

UniApp Android原生插件开发 极简讲解

UniApp Android原生插件开发 极简讲解

UniApp的Android原生插件，核心是通过Java/Kotlin编写符合UniApp规范的代码，封装Android原生能力（如调用系统API、硬件交互等），让UniApp能调用原生功能。以下是核心步骤和关键点：

一、开发前准备

1. 环境搭建：

- 安装Android Studio（需配置JDK、Android SDK，建议对应UniApp要求的版本）；
- 安装HBuilderX（用于后续插件调试、打包）；
- 了解UniApp插件规范：UniApp原生插件基于Android的aar/jar包，需遵循其接口定义规则。

2. 核心概念：

- 插件入口类：必须继承 `io.dcloud.feature.uniapp.common.UniModule`；
- 方法注解：暴露给UniApp调用的方法需加 `@UniJSMethod(uiThread = boolean)`（`uiThread=true`表示在主线程执行，`false`在子线程）；
- 回调UniApp：通过 `UniJSCallback` 实现原生向JS层传值（成功/失败回调）。

二、核心开发步骤（以Java为例）

1. 创建Android Library项目

- 打开Android Studio → 新建Module → 选择「Android Library」，命名（如 `uni_plugin_demo`）；
- 引入UniApp核心依赖：在Module的`build.gradle`中添加（需对应UniApp版本）：

Code block

```
1 dependencies {
2     // UniApp核心依赖（可从HBuilderX安装目录获取）
3     implementation files('libs/uniapp-v8-release.aar')
4     // 基础依赖
5     implementation 'androidx.appcompat:appcompat:1.4.1'
6 }
```

2. 编写插件核心类

创建自定义Module类，继承UniModule，编写暴露给JS的方法：

Code block

```
1  import io.dcloud.feature.uniapp.common.UniModule;
2  import io.dcloud.feature.uniapp.annotation.UniJSMETHOD;
3  import io.dcloud.feature.uniapp.bridge.UniJSCallback;
4
5  public class MyUniPlugin extends UniModule {
6
7      // 示例1: 同步方法 (无回调)
8      @UniJSMETHOD(uiThread = false)
9      public String getAndroidVersion() {
10         return android.os.Build.VERSION.RELEASE; // 返回Android系统版本
11     }
12
13     // 示例2: 异步方法 (带回调)
14     @UniJSMETHOD(uiThread = true)
15     public void showToast(String msg, UniJSCallback callback) {
16         if (mUniSDKInstance == null) { // mUniSDKInstance是UniModule内置的上下文
对象
17             if (callback != null) {
18                 callback.invoke(new HashMap<String, Object>() {{
19                     put("code", -1);
20                     put("msg", "上下文为空");
21                 }});
22             }
23             return;
24         }
25         // 原生Toast
26         Toast.makeText(mUniSDKInstance.getContext(), msg,
Toast.LENGTH_SHORT).show();
27         // 回调JS层
28         if (callback != null) {
29             callback.invoke(new HashMap<String, Object>() {{
30                 put("code", 0);
31                 put("msg", "Toast显示成功");
32             }});
33         }
34     }
35 }
```

3. 配置插件清单

在Module的 `src/main/assets/dcloud_uniplugins.json` 中声明插件（UniApp识别插件的关键）：

```

1  code block
2  "plugins": [
3    {
4      "type": "module",
5      "name": "MyUniPlugin", // 插件名 (JS调用时用)
6      "class": "com.demo.MyUniPlugin" // 插件类的完整包名+类名
7    }
8  ]
9  }

```

4. 打包插件

- 在Android Studio中执行「Build → Make Module 'uni_plugin_demo」，生成aar包（路径：module/build/outputs/aar/xxx-release.aar）；
- 将aar包和dcloud_uniplugins.json放到UniApp项目的
nativeplugins/MyUniPlugin/android/ 目录下（需手动创建目录结构）。

三、UniApp中调用插件

- 在UniApp项目的pages.json中配置插件：

Code block

```

1  {
2    "plugins": {
3      "MyUniPlugin": {
4        "version": "1.0.0",
5        "provider": "开发者ID（本地调试可随便填）"
6      }
7    }
8  }

```

- 在JS代码中调用：

Code block

```

1  // 获取插件实例
2  const myPlugin = uni.requireNativePlugin('MyUniPlugin');
3
4  // 调用同步方法
5  const androidVersion = myPlugin.getAndroidVersion();
6  console.log("Android版本: ", androidVersion);
7
8  // 调用异步方法（带回调）
9  myPlugin.showToast("Hello UniApp", (res) => {

```

```
10     console.log("回调结果:", res);  
11 });
```

四、关键注意事项

1. **上下文获取**：优先用 `mUniSDKInstance.getContext()`（UniApp的上下文），避免用 `Application Context` 导致弹窗、页面跳转异常；
2. **线程问题**：耗时操作（如网络请求、文件读写）需标注 `uiThread = false`，避免阻塞主线程；
3. **调试方式**：
 - 本地调试：将UniApp项目运行到Android真机/模拟器，HBuilderX会自动打包插件到应用中；
 - 日志调试：用Android Studio的Logcat查看原生日志（需打Tag）；
4. **权限处理**：若插件涉及权限（如相机、存储），需在UniApp的manifest.json中声明，或在原生代码中动态申请。

总结

Android原生插件开发的核心是「遵循UniApp规范封装原生方法→打包aar→在UniApp中配置并调用」，关键是做好接口定义、上下文管理和回调处理，适合扩展UniApp无法直接实现的原生能力（如硬件交互、系统级API调用等）。

（注：文档部分内容可能由 AI 生成）